# COMPARATIVE STUDY OF TRADITIONAL SOFTWARE DEVELOPMENT AND LOW-CODE DEVELOPMENT PLATFORMS

*Sai Pranav B V*

*241401088*

## Abstract

Software development has evolved from traditional coding methods to modern low-code platforms that simplify application development. Traditional software development requires developers to write code manually, offering greater flexibility and customization but requiring more time, skilled professionals, and higher costs. In contrast

Low-Code Development Platforms use visual interfaces, drag-and-drop tools, and pre-built components to develop applications quickly with minimal coding. This report compares Traditional Software Development and Low-Code Development Platforms based on their development process, architecture, advantages, disadvantages, performance, scalability, security, cost, and real-world applications.

### Traditional Software Development

#### Definition

Traditional Software Development is the process of building software by writing source code manually using programming languages.

Developers create every component including:

- User Interface
- Business Logic
- Database Connectivity
- Security
- APIs
- Testing Scripts

Examples of programming languages include:

- C
- C++
- Java
- Python
- C#

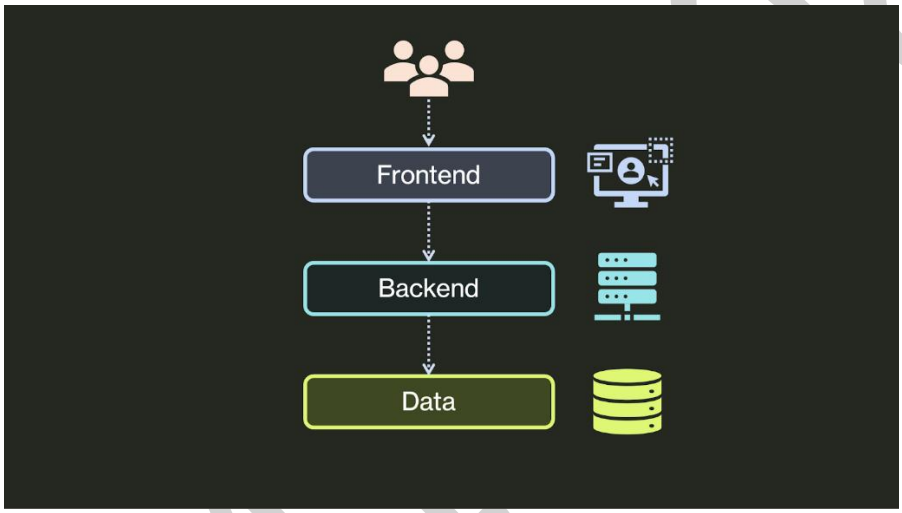
241401088

- PHP
- JavaScript

Frameworks include:

- Spring Boot
- Django
- ASP.NET
- Laravel
- React
- Angular

**Architecture**




**Development Process**

Traditional development follows the Software Development Life Cycle (SDLC).

**Requirement Analysis**

Developers gather customer requirements.

Example:

Customer wants an Online Shopping System.

Requirements:

- Login
- Product Search
- Cart

- Payment
- Orders

**System Design**

Architects design

- Database
- Software Architecture
- APIs
- UI Wireframes

**Testing**

Testing includes

- Unit Testing
- Integration Testing
- System Testing
- Performance Testing
- Security Testing

**Deployment**

Deploy software on

- AWS
- Azure
- Google Cloud
- On-premise Servers

**Maintenance**

After deployment,

- Bug fixing
- Feature updates
- Performance optimization
- Security patches

**Advantages**

2140108

**Complete Flexibility**

Developers can build any functionality.

Example

Artificial Intelligence

Machine Learning

Gaming Engines

Operating Systems

**High Performance**

Applications can be optimized for

- Speed
- Memory
- CPU utilization

**Better Security**

Developers control

- Encryption
- Authentication
- Authorization

**Unlimited Customization**

Every feature can be customized.

**Disadvantages**

Long development time.

High cost.

Requires skilled developers.

Large development teams.

Complex maintenance.

## Low-Code Development Platform

**Definition**

2140108

A Low-Code Development Platform allows software applications to be built using graphical interfaces instead of extensive coding.

Developers use

- Drag and Drop Components
- Visual Workflows
- Ready-made Templates
- Automatic Code Generation

Only small amounts of custom code are required.

**Popular Platforms**

- Microsoft Power Apps
- Mendix
- OutSystems
- Zoho Creator
- Appian
- Salesforce Lightning
- Oracle APEX

**Architecture**




140108

**Development Process**

**Step 1**

Choose application template.

↓

**Step 2**

Drag UI components.

↓

**Step 3**

Configure workflow.

↓

**Step 4**

Connect database.

↓

**Step 5**

Deploy application.

**Features**

Visual Development

Reusable Components

Cloud Deployment

Responsive UI

Workflow Automation

API Integration

**Advantages**

**Faster Development**

Applications can be built in days instead of months.

**Lower Cost**

Fewer developers are needed.

**Easy Maintenance**

2414O1088

Updates are made visually.

**Citizen Development**

Business users can develop applications.

**Detailed Comparison**

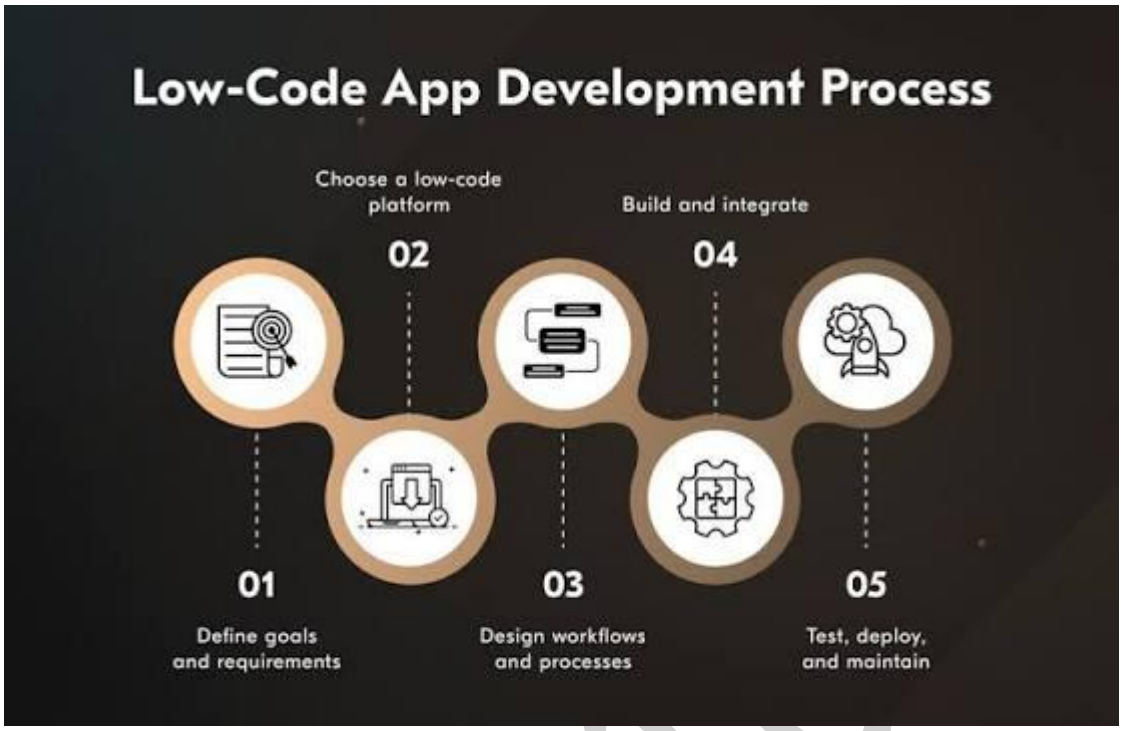
| Feature | Traditional Development | Low-Code Platform |
|---|---|---|
| Coding | 100% Manual | Minimal Coding |
| Development Speed | Slow | Very Fast |
| Learning Curve | High | Low |
| Cost | High | Lower |
| Team Size | Large | Small |
| Customization | Unlimited | Limited |
| Security Control | Full | Platform Dependent |
| Performance | Excellent | Good |
| Scalability | Very High | Moderate to High |
| Maintenance | Complex | Easy |
| Deployment | Manual | Automated |

**Traditional SDLC**




24140088

**Low-Code SDLC**




## Challenges

**Traditional Development Challenges**

Long development cycles.

High maintenance costs.

Talent shortage.

Project delays.

Complex debugging.

**Low-Code Challenges**

Vendor dependency.

Licensing expenses.

Limited flexibility.

Performance bottlenecks.

Complex integration in some scenarios.

241401